Reflection: Miller-Rabin (Brad)

What's the worst case for your algorithm, and what kind of input triggers it?

The worst case is a large prime, but two things drive the cost.

First, a prime can never be rejected. A composite stops as soon as one round finds a witness (usually the first round), but a prime has no witness, so it has to survive all k rounds before we accept it.

Second, each round is a modular exponentiation. For a b-bit n that is about b squarings plus up to b/2 multiplications, so the total cost is O(k · b) multiplications. The cost is linear in k, not exponential: doubling k doubles the time. What falls exponentially is the error, which is at most 4^-k. So a larger k buys much more certainty for only a proportional increase in work, but it is always paid in full by primes.

What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest part was ensuring that my algorithm was right, because even though I had my expectations from the algorithm the amount of data that was prepared meant that small inefficiencies or differences in runtime could become apparently large in practice, but also if I messed up edge cases this would show small errors that wouldn't be obvious from the time complexity

Checking all of that was difficult, as I'm not confident enough in the maths to spot when a result is subtly wrong, so I used Claude to help verify the code by feeding it a wikipedia article and getting it to compare against the pseudocode. This was really beneficial as I also like to get things phrased in a simpler way, which I can then expand on once I know my base understanding is correct to nail things like the terminology, edge cases, and other technical understanding.

If you did this implementation again from scratch, what would you do differently?

I would build it in small steps and not move on until I could explain each one without notes, instead of kind of hacking away at it slowley and fixing things up. But that was also due to time constraints as I started later than I wanted too and was scared of the scope. This would also mean non-critical things like edge cases last, ideally. I jumped between the maths and the code, which is why the squaring chain took so long to click.

What's one thing about your algorithm that you didn't fully understand before starting, and now do?

The composite certainty/prime uncertainty. It took a bit to get that getting a witness is the proof. If the chain reaches a 1 when the previous result was 1 nor -1, which isn't valid when modulo a prime, so n is composite. I think it was simply a depth thing, as it took a bit of math myself and playing around with the relation $x^2 \equiv 1 \pmod{n}$, where a prime only has 1 and -1 as square roots, but a composite can have others, to get it to finally click why this rule was correct and we could run our algorithm based on it.